CONFIDENTIAL

Question 1

instructions:

- No starter code is provided for this question . You will have to write the program from scratch.
- As for question (a), write a complete C++ program and save it in a single file named **prerequisiteDSA_yourname.cpp**

Consider the UML class diagram in Figure 2 and answer the following questions:

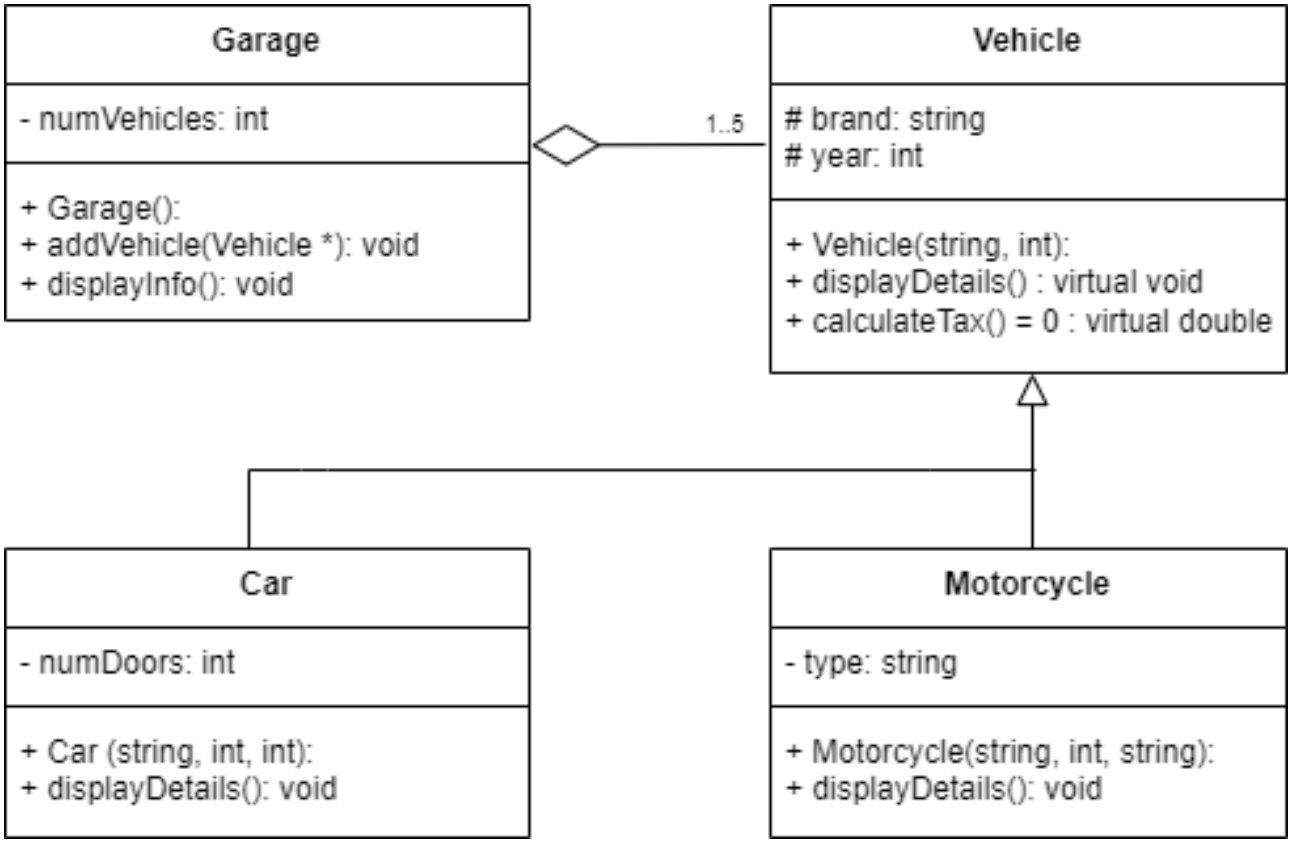


Figure 2: UML class diagram

a. Write a complete C++ program based on the UML class diagram in **Figure 2**. Implement all the classes with their member variables (attributes) and member functions (methods) as shown in the diagram. The purpose of each function is as the name implies, and more explanations are provided below. Your program should be able to generate the output shown in **Figure 3**. Tasks to be accomplished for the program are as follows:

i. Define the class **`Vehicle`**. The cl ass has three member functions:

- Constructor with arguments: to initialize all the member attributes for the class.
- **`displayDetails()`**: to display a vehicle's brand and year. Please allow the function to be polymorphic.
- **`calculateTax()`** is a pure virtual function.

(7.5 marks)

ii. Define the class **`Motorcycle`** as a child of the class **`Vehicle`** . The class has three member functions:

- Constructor with arguments: to initialize all the member attributes for the class, including the parent's attributes.
- **`displayDetails()`**: to display a motorcycle's type, brand and year. The function should invoke the parent's **`displayDetails()`** function.
- **`calculateTax()`**: to return the motorcycle's tax, which is MYR50.

(8.5 marks)

iii. Define the class **`Car`** as a child of the class **`Vehicle`**. The class has three member functions:

- Constructor with arguments: to initialize all the member attributes for the class, including the parent's attributes.
- **`displayDetails()`**: to display a car's type, brand and the number of doors. The function should invoke the parent's **`displayDetails`** function.
- **`calculateTax()`**: to return the car's tax. The tax is calculated using the formula: 100 + (number of doors × 50).

(9 marks)

iv. Define the class **`Garage`**. The class has three member functions:

- Default constructor: to initialize all the member attributes for the class with suitable values.
- **`AddVehicle():`** to assign the element in the array of **`Vehicle`** pointers with the passed argument. It also updates the number of vehicles.
- **`DisplayInfo()`**: to display the details of the vehicles in the garage.

(12.5 marks)

v. Define the **`main`** function:

- Create a **`Garage`** object.
- Create an array of **`Vehicle`** pointers that dynamically allocates cars and motorcycles.Use sample data provided in the file **question2_data.txt**
- Add all the created vehicles to the **`Garage`** object.
- Display the details of all the vehicles in the garage.

(9.5 marks)

vi. Add appropriate statements in the method `AddVehicle()` of class `Garage` to handle the error if the garage is full using an exception approach. **Figure 4** shows the example of program output when the garage is full. (2 marks)

```
***** Garage Details *****

1. Car Details:
   Brand: Toyota
   Year : 2015
   Number of Doors: 4
   Tax  : MYR300
---------------------------------

2. Car Details:
   Brand: BMW
   Year : 2019
   Number of Doors: 2
   Tax  : MYR200
---------------------------------

3. Motorcycle Details:
   Brand: Honda
   Year : 2020
   Type : Sport
   Tax  : MYR50
---------------------------------

4. Car Details:
   Brand: Nissan
   Year : 2018
   Number of Doors: 5
   Tax  : MYR350
---------------------------------

5. Motorcycle Details:
   Brand: Harley-Davidson
   Year : 2017
   Type : Cruiser
   Tax  : MYR50
---------------------------------
```

Figure 3: Program output

```
An error occurred: The maximum number of vehicles has been
reached!!
```

Figure 4: Sample output of error handling